Yalla at AISA-ArabicFC: Argument-Focused Fine-Tuning and Per-Argument Ensembling for Arabic Function Calling

Manish Joshi
Independent Researcher
manishinuae@gmail.com

Abstract

We describe the Team Yalla submission to the AISA-ArabicFC shared task on Arabic function calling, which ranked second in both tracks (0.9273 Track A, 0.9395 Track B), within 0.0006 of the winning system. The headline metric, exact argument match (ARGEM), is unforgiving: a single wrong surface form loses the whole example. The provided targets bury the scored object inside a rationale: the tool name and argument dictionary are only 30.3% of the target characters. We trained on the 30.3%, which is our largest single gain (+0.020 ARGEM in a matched comparison). A matched follow-up shows that reasoning need not be removed entirely: emitting it after the arguments trails the JSON-only target by only 0.008 ARGEM. We add a reinforcement-learning stage that uses the official scorer as the reward, and combine three deliberately diverse fine-tunes with a per-argument majority vote, which repairs individual argument slots rather than choosing whole predictions. Every member is a LoRA fine-tune of a public 4B or 7B Qwen model trained on the official split alone, with no distillation and no external data. We note that two of the three voting members emit no rationale, so our Track B `think` field is attached after prediction from a separate reasoning-trained model whose predicted call for that row is identical: it is a post-hoc explanation, not the computation that produced the arguments. We report the experiments that failed alongside those that worked.

1 Introduction

Function calling asks a model to map a natural-language request onto a typed API call: decide whether any of the candidate tools applies, select one, and extract its arguments as JSON (Patil et al., 2023; Qin et al., 2024). The AISA-ArabicFC shared task (Nacar et al., 2026a) poses this problem for Arabic, including dialectal Arabic, with a deliberately strict headline metric: ARGEM counts an example only if the predicted argument dictionary matches the gold dictionary *exactly* after normalization. There is no partial credit; copying a name verbatim where the gold transliterates it, or resolving a relative date that the gold keeps as the uttered phrase, scores zero for the example.

Tool selection saturates quickly under fine-tuning (our final test FNACC is 0.9973, three errors in 1,125 rows), so what is left to win is argument surface forms. That is where the whole budget went.[1] We contribute four things. An **argument-focused objective** strips the reasoning field from the supervised target so that all of the loss lands on the scored JSON, worth +0.020 ARGEM in a matched pair and our largest single lever. A **placement analysis** shows most of that gain survives without giving up a per-example rationale: emitting it after the arguments costs only 0.008 (Section 4). A **GRPO stage** (Shao et al., 2024) takes its reward from the official normalizer's own exact-match check rather than a proxy (Section 5). A **per-argument vote** lets three diverse fine-tunes assemble a correct dictionary from models that are each wrong somewhere (Section 3). We also report what did not work. Two caveats apply throughout: the final ensemble was selected on the development set, and most ablations are single runs.

2 Task and Data

Each example provides an Arabic query and four candidate tool schemas; the system outputs `tool_called`, a candidate name or `none`, and `arguments`. The data and the surrounding agentic framework are described by Nacar et al. (2026b) and Nacar et al. (2026c). Track B additionally requires a non-empty Arabic reasoning field (`think`). Track A is scored as $0.4\,\text{FNACC} + 0.6\,\text{ARGEM}$ and Track B as $0.3\,\text{FNACC} + 0.5\,\text{ARGEM} +$

[1] Code, models and reproduction instructions: https://github.com/mcjoshi/ArabicFC

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1608–1613
October 28-29, 2026 ©2026 Association for Computational Linguistics

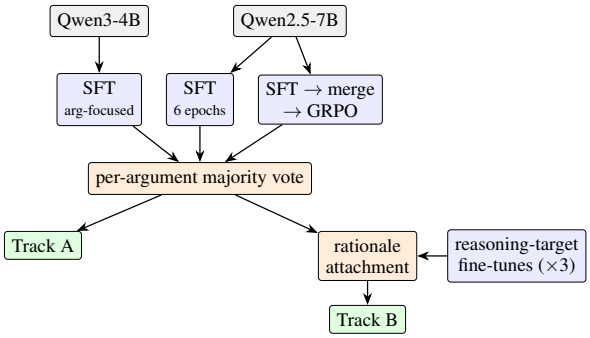


Figure 1: System overview. Three diverse LoRA fine-tunes vote per argument key. Because two members emit no rationale, the Track B `think` field is attached after prediction from a reasoning-trained model whose prediction for that row is identical; arguments are never modified by this step.

0.2 THINKRATE. The data comprise 10,550 training (50 negative), 545 development (45 negative), and 1,125 blind test examples across Modern Standard Arabic and Gulf, Levantine, Egyptian, and Maghrebi dialects. For experiments that needed a second held-out set we carved 1,007 rows out of the training split, leaving 9,543 for training; we refer to this as the validation split, and no model trained on the remaining 9,543 rows ever saw it. Annotation policy dictates that arguments are included only when explicitly stated by the user, and revised gold conventions keep many values verbatim as uttered (e.g. a relative date is kept as the phrase, not resolved to ISO format), which makes surface-form fidelity, rather than semantic correctness, the binding constraint.

3 System

Figure 1 shows the pipeline. All members are LoRA fine-tunes (Hu et al., 2022) (r=32 or 64, α=2r) of public Qwen bases (Qwen Team, 2024, 2025), trained on the official training split only and decoded greedily with vLLM (Kwon et al., 2023). The three voting members are chosen for diversity along every axis we could vary cheaply: base family and size (Qwen3-4B vs. Qwen2.5-7B), training length (3 vs. 6 epochs), and objective (supervised only vs. an added RL stage).

Per-argument vote. For each example, the plurality tool is selected first; among the members selecting that tool, each argument key is included if at least half of them (rounded up) produced it, and its plurality value is used. Voting per key instead of per prediction lets the ensemble build a fully correct dictionary from members that each err on different slots, which is what an all-or-nothing metric rewards. Value ties are resolved by member order, making order part of the recipe (Appendix B).

Track B rationale. Two of the three voting members are trained with the argument-focused objective and so emit no rationale, and we therefore had to supply the `think` field separately. We held this step to two rules. First, every rationale is produced by one of our own reasoning-trained fine-tunes. Second, a rationale is reused for a row only when that model's predicted tool and arguments are *identical* to what we submit, so it can never describe a call we do not make; this covers 97.7% of rows, and the remaining 2.3% receive a row-specific description derived from the submitted call itself. The step never modifies `tool_called` or `arguments`. We record per-row provenance for all 1,125 rows and release it with a verifier script. The procedure produces a distinct rationale for each example. It is a post-hoc explanation, not the reasoning that produced the arguments. Section 4 measures what a single model doing both would cost.

4 Argument-Focused Training

The distributed targets have the form `<think>` rationale `</think>` JSON. ARGEM is decided entirely by the serialized call $z = \{\texttt{tool_called}, \texttt{arguments}\}$, so that is what we supervise. Standard fine-tuning with assistant-only loss instead minimizes cross-entropy over the full target $y = [r; z]$, a rationale r followed by z, of which z is only 30.3% of the characters. Our change is confined to the target: we keep the prompt masking and drop r,

$$\mathcal{L}(\theta) = -\sum_{t=1}^{|z|} \log p_\theta(z_t \mid x,\, z_{<t})\,, \qquad (1)$$

so that every supervised token belongs to the scored object $z = \{\texttt{tool_called}, \texttt{arguments}\}$. Architecture, data, and decoding are untouched.

Table 1 shows the effect in matched pairs; training details are in Table 5 (Appendix A). Removing the rationale from the target is worth +0.020 ARGEM on the development set. This result, however, conflates two changes: (i) the loss budget shifts entirely to scored tokens, and (ii) at inference the argument tokens are no longer conditioned on a sampled reasoning prefix, removing a source of

Target format	Split	ARGEM
reasoning → JSON	dev	0.8440
JSON only (arg-focused)	dev	**0.8640**
JSON only (arg-focused)	val	**0.8352**
JSON → reasoning	val	0.8272

Table 1: Target-format ablation. Each pair is matched (Qwen3-4B, LoRA r=32, 3 epochs, identical training rows); the two pairs use different data subsets and evaluation splits, so they are comparable within, not across, pairs. Single runs.

drift between what the model says and what it extracts. Chain-of-thought prompting and sampling-based aggregation (Wei et al., 2022; Wang et al., 2023) improve reasoning-heavy tasks, but the interaction between generated rationales and strict structured output is less settled, which is what motivates the following ablation. To probe this we trained a third arm whose target is a single JSON object with the rationale as its *final* field: arguments are generated first, then the explanation. Putting the rationale last costs only 0.008 ARGEM against the JSON-only target (0.8272 vs. 0.8352 on our validation split), and still yields a genuine per-example rationale. Rationale-first and rationale-last were never compared in a single fully matched three-arm experiment, so we cannot cleanly separate the conditioning explanation from the loss-dilution one. The pair we do have does show that, on this task, most of the benefit is available without giving up a per-example rationale.

5 Scorer-in-the-Loop RL

Greedy decoding often misses surface forms that the model can produce: on development data an oracle over eight samples reached roughly 0.92 ARGEM against 0.85 for greedy. We therefore add a GRPO stage (Shao et al., 2024): the supervised LoRA is merged into the base weights, since a second adapter cannot be trained cleanly on top of a first, and a fresh LoRA is trained as the policy. For each prompt we sample G=8 completions at temperature 1.0 and score each with

$$r(y) = \begin{cases} 1 & \text{exact} \\ s(y) & \text{otherwise,} \end{cases} \qquad (2)$$

where *exact* means the tool is correct and the arguments match under the *official* scorer's comparison, and the shaping term $s(y) = 0.3\mathbb{1}[\text{tool}] + 0.1\mathbb{1}[\text{JSON}] + 0.1\mathbb{1}[\text{rationale}]$ gives partial credit.

Member	dev ARGEM
SFT, arg-focused, Qwen3-4B	0.8640
SFT, 6 epochs, Qwen2.5-7B	0.8420
SFT+GRPO, Qwen2.5-7B	0.8640
Per-argument ensemble	**0.8780**

Table 2: The submitted ensemble against its members on the development set. The vote beats its best member while containing the weakest.

	FNACC	ARGEM	Overall	Rank
Dev (Track A)	1.0000	0.8780	0.9268	–
Test (Track A)	0.9973	0.8806	0.9273	2
Test (Track B)	0.9973	0.8806	0.9395	2

Table 3: Final results. The winning systems scored 0.9279 (A) and 0.9400 (B); the gap to first place is roughly one test example's contribution.

Rewards are standardized within the group to obtain advantages, so no value network is needed. Rewarding the real metric matters: the normalizer defines equivalence classes (digit systems, boolean forms, list separators) that a generic string reward would miss, and the shaping term supplies gradient on near misses. In single runs GRPO moved ARGEM from 0.860 to 0.864 at 7B, the member we use, from 0.854 to 0.866 at 3B, and left 4B unchanged; these differences are small, unreplicated, and do not establish a scale trend. The stage earns its place less through that margin than by contributing a model whose errors differ from its supervised siblings, though we did not run a leave-one-out ensemble to confirm it.

6 Results

Table 2 sets the ensemble against its members: +0.014 over the best of them on development data, while containing the weakest. That margin is measured on the split the ensemble was selected on, and should be read as optimistic. The vote can nonetheless assemble a dictionary that no single member produced. Table 3 shows the final scores: second place in both tracks, 0.0006 and 0.0005 behind the winner. Three call/no-call errors cost us 0.0011 on Track A, more than that margin: a task where FNACC looks solved was still decided by FNACC. Test ARGEM (0.8806) and development ARGEM (0.8780) differ by 0.0026, which is well inside the variation we measure between reruns and does not by itself demonstrate generalization; what it shows is that no collapse occurred between the two splits. The winning combination was the best

of 2,508 member-subset and voting-rule combinations evaluated on 545 development examples, so its +0.014 margin over the best single member is an optimistic estimate, and we did not score individual members on the blind test. A useful control: a variant trained on train+dev reached 0.9260 development ARGEM, but a controlled pair showed +0.076 of that to be memorization, and on the blind test it did not beat the ensemble described here. The train-only ensemble remained our best system.

Dialect	n	n_+	FNACC	ARGEM
MSA	323	289	1.0000	0.9135
Gulf	75	72	1.0000	0.8750
Levantine	75	67	1.0000	0.7761
Egyptian	66	66	1.0000	0.8182
Maghrebi	6	6	1.0000	1.0000

Table 4: Per-dialect development results for the submitted ensemble (official scorer breakdown). FNACC is computed over all n rows and ARGEM over the n_+ positive rows only. The Maghrebi cell has six examples and should not be interpreted.

Dialects. We made no dialect-specific design choices; all members are trained uniformly on the mixed-dialect data. Table 4 reports the official per-dialect breakdown of our ensemble on the development set. Tool selection is uniform across dialects, but ARGEM is not: Modern Standard Arabic reaches 0.9135 while Levantine trails at 0.7761, and the official dialect-gap metric is 0.2239 (inflated by the six-example Maghrebi cell). Since FNACC shows no gap, the disparity lies in argument extraction and value rendering, not in tool selection. Where our three ensemble models disagree with each other, the disagreements are of exactly this kind: a date written with Arabic-Indic digits against the same date in ASCII digits, a dialectal time expression kept verbatim against its Modern Standard Arabic or English equivalent, a noun with and without the definite article *al-*, and two spellings of the same word that differ only in a diacritic. Each is a single character-level choice that the metric treats as a total loss. On Levantine the weakest member overall (0.8420) becomes the strongest (0.8060 vs. 0.7612 and 0.7463). We did not plan for that, but it is exactly the kind of disagreement the per-argument vote exists to cash in. The organizers' robustness ranking, a macro-average of per-dialect Overall, places us fifth with the widest maximum-minus-minimum gap, 0.140: among the five ranked teams we are first on Modern Standard Arabic and on Levantine, last on Gulf, and joint-last on Maghrebi. The width comes from the floor, not the ceiling, since our best dialect is 0.011 above the leading team's best while our worst is 0.053 below theirs. Which dialect is weakest is less clear, as Levantine is our worst on development data and our joint-best on test; we claim only that the spread is real and unaddressed. Closing it, for instance with dialect-balanced or dialect-augmented training, is the largest remaining gain we can identify.

What did not work. Measured and discarded: retrieval-augmented prompting at inference (-0.020 ARGEM), frontier-model distillation, deterministic post-processing, higher LoRA rank, longer SFT, longer or repeated RL, a binary RL reward, and adding the arguments-first model to the ensemble (its errors correlate with the supervised members').

Reproducibility. Decoding is greedy, but batched GPU inference is not bit-deterministic, so a rerun does not reproduce a submission file exactly. Three independent end-to-end reruns of our pipeline agreed with the submitted file on 99.0 to 99.5% of rows, and with each other over the same range, so the submitted file is an ordinary sample of this process rather than an outlier; tool selection matched on 100% of rows in every run. The organizers independently verified our submission from the weights and scripts we supplied, which we release publicly: code and training scripts at `https://github.com/mcjoshi/ArabicFC`, adapters and the merged base at `https://huggingface.co/mojo28/aisa-arabicfc-ens-v3`.

7 Conclusion

We never found a change of scale or architecture that mattered as much as where the training signal lands. Deleting the rationale from the target was worth more (+0.020) than an entire RL stage. The arguments-first ablation suggests systems can keep genuine rationales at a modest cost (0.008 ARGEM) by emitting them after the answer, a simple recipe for tasks where reasoning is wanted but strict output fidelity is scored.

Limitations

Most ablations are single training runs; the key comparison in Table 1 is matched but unreplicated,

and differences of 0.008 to 0.020 ARGEM correspond to roughly 8 to 10 examples. The final ensemble was the best of 2,508 candidates on the 545-example development set, and we did not score individual members on the blind test, so its margin over a single model is optimistic. The Track B rationale is a post-hoc explanation attached after prediction, and ensembling triples inference cost. Two omissions cost us more than any technique gained. Negative examples are 0.5% of training but 8.3% of development data, and three call/no-call errors on the test set were worth 0.0011 on Track A, more than the 0.0006 that separated us from first place. Bringing every dialect to the Modern Standard Arabic level would add roughly 0.037 ARGEM, though that rests on development cells as small as six examples. Finally, batched GPU inference is not bit-deterministic, so re-running our pipeline reproduces 99.0 to 99.5% of the submitted rows rather than all of them.

Acknowledgments

We thank the AISA-ArabicFC organizers for a well-run task, for the open scorer that made scorer-in-the-loop training possible, and for their reproducibility verification.

References

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The first shared task on Arabic function calling for agentic AI systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From language to action in Arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

	SFT	**GRPO**
initialization	public base	merged SFT model
LoRA rank r	32 / 64	32
LoRA α / dropout	$2r$ / 0.05	$2r$ / 0.05
learning rate	2×10^{-4}	1×10^{-6}
schedule	cosine, 3% warmup	constant, 3% warmup
effective batch	32 targets	4 prompts ($\times8$)
epochs / steps	3 (6 for one member)	400 steps
sampling	–	temperature 1.0
max length	1,536	1,280 + 320
precision	bf16	bf16

Table 5: Hyperparameters. LoRA is applied to all attention and MLP projections. SFT uses assistant-only loss (prompt tokens masked). Decoding is greedy (temperature 0) in vLLM with at most 512 new tokens for every reported result.

Omer Nacar, A. Deema, and A. Mohammed. 2026c. AISA: A unified architecture for agentic AI systems.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

A Training and Decoding Details

The rank-64 member is the 6-epoch Qwen2.5-7B fine-tune; all other members use rank 32. The GRPO stage samples roughly 1,600 prompts from

the training split (4 prompts per optimizer step for 400 steps, 8 completions each).

B The Per-Argument Vote

Given member predictions $\{(t_m, a_m)\}_{m=1}^{n}$ for one example: (1) the final tool t^* is the plurality of $\{t_m\}$; if t^* is none, the arguments are empty; (2) otherwise, restrict to the members with $t_m = t^*$, and include argument key k if and only if at least half of them (rounded up) produce k; (3) the value for k is the plurality value among those members. Ties in steps (1) and (3) are resolved by member order, which therefore forms part of the recipe; we use the submitted order (arg-focused 4B, 6-epoch 7B, GRPO 7B).